Universidad Autónoma de Sinaloa

Facultad de Ingeniería Los Mochis

Materia:

“Software de Sistemas”

Trabajo:

“Desarrollo de Funciones”

Profesor:

Edgar Omar Pérez Contreras

Integrantes:

Laurean Acosta Luis Donaldo

Rosas Castro Manuel Ernesto

Fecha:

29/01/2026

Función 1:

- **Nombre:** preprocesarArchivo
- **Descripción**: La función abre un archivo de texto y lo recorre para buscar y eliminar los siguientes elementos: Espacios en Blanco, Líneas Vacías, Comentarios, Tabuladores. Una vez que le borre todos esos elementos debe guardar los cambios realizados en otro archivo
- **Notas:** Debe validar si el archivo existe, si no existe marca un error. Una vez que termina de leer el archivo, debe cerrarlo

Código:

```
//  Función 1: preprocesarArchivo
void preprocesarArchivo(string nombreEntrada, string nombreSalida) {
    ifstream archivoLectura(nombreEntrada);
    ofstream archivoEscritura(nombreSalida);

    //  Validación de existencia
    if (!archivoLectura.is_open()) {
        cerr << "Error: El archivo de entrada '" << nombreEntrada << "' no existe o no se pudo abrir." << endl;
        return;
    }

    string linea;
    bool enComentarioBloque = false;

    while (getline(archivoLectura, linea)) {
        string lineaProcesada = "";

        for (size_t i = 0; i < linea.length(); ++i) {
            //  Manejo de comentarios de una sola línea (//)
            if (linea[i] == '/' && i + 1 < linea.length() && linea[i + 1] == '/') {
                break;
            }

            // Filtrar espacios, tabuladores y saltos de línea
            if (!isspace(linea[i])) {
                lineaProcesada += linea[i];
            }
        }

        //  Solo escribir si la línea resultante no está vacía
        if (!lineaProcesada.empty()) {
            archivoEscritura << lineaProcesada << endl;
        }
    }

    archivoLectura.close();
    archivoEscritura.close();
    cout << "Archivo preprocesado exitosamente en: " << nombreSalida << endl;
}
```

Terminal salida:

```
Archivo preprocesado exitosamente en: archivo_limpio.txt
PS C:\Users\LaptopLuis\Programas>
```

Archivo de entrada:

```
C++ > codigo_fuente.txt
1   Posicion = valor + inical   * 60
```

Archivo de salida:

```
deSnap    archivo_limpio.txt
> archivo_limpio.txt
Posicion=valor+inical*60
```

Función 2:

- **Nombre:** recorrerArchivo
- **Descripción**: La función abre un archivo de texto previamente preprocesado por la función preprocesarArchivo y lo recorre carácter por carácter e imprime en pantalla el carácter leído
- **Notas:** Debe validar si el archivo existe, si no existe marca un error. Una vez que termina de leer el archivo, debe cerrarlo

Código:

```
//  Función 2: recorrerArchivo
void recorrerArchivo(string nombreArchivo) {
    ifstream archivo(nombreArchivo);
    char caracter;

    if (!archivo.is_open()) {
        cerr << "Error: El archivo '" << nombreArchivo << "' no existe." << endl;
        return;
    }

    cout << "Contenido del archivo caracter por caracter:" << endl;
    while (archivo.get(caracter)) {
        cout << "[" << caracter << "]";
    }
    cout << endl;

    archivo.close();
}
```

Terminal de salida:

```
Archivo preprocesado exitosamente en: archivo_limpio.txt

Contenido limpio (sin saltos de linea):
[H][o][l][a][c][o][m][o][e][s][t][a][s]
PS C:\Users\LaptopLuis\Programas>
```

Archivo de entrada:

```
C++ > codigo_fuente.txt
1   Hola como estas
```

Archivo de salida:

```
C++ > archivo_limpio.txt
1   Holacomoestas
```

Conclusión y Comentarios

Laurean Acosta Luis Donaldo: El haber realizado el trabajo de estas funciones me permitió comprender lo que no se ve dentro de la programación. Me resultó fascinante notar que la limpieza léxica es un pilar fundamental de casi cualquier compilador; antes de ejecutar una sola instrucción, el sistema debe transformar nuestro código legible en un flujo de datos puro y sin ambigüedades, eliminando todo aquello que no aporte valor lógico a la máquina, como espacios y comentarios.

Rosas Castro Manuel Ernesto: Algo que si note y se me complico fue el hecho de que en la segunda función (**recorrerArchivo**) cuando lo ejecutaba en la terminal de salida se mostraba un corchete cerrado vacío (Ejemplo = []), entonces me di cuenta que lo que sucede es que en los archivos de texto existen caracteres invisibles que, aunque tú no los veas en el editor, el programa sí los lee y la solución que encontramos fue la siguiente:

```
// Esta condicion evita que los saltos de linea (\n o \r) se impriman
        if (caracter != '\n' && caracter != '\r') {
            cout << "[" << caracter << "]";
        }
```